

Tide retrospective: κ_3 vanishing for affine observables (harmonic Gibbs)

Daniel Murfet

7 May 2026

Abstract

A small *strict-improvement* tide of Tide 5 (`kappa3_harmonic_id_id_id_eq_zero`): generalise from the identity-observable case to an arbitrary triple of affine observables. For the 1D harmonic potential $L(x) = (\lambda/2)x^2$ with $\lambda, t > 0$,

$$\kappa_3(b_1x + c_1, b_2x + c_2, b_3x + c_3) = 0$$

under the harmonic Gibbs measure on $\mathbb{R}$. The proof goes by *moment expansion* (per the GPT-5.5 Pro consult): each of the seven Gibbs expectations in the cumulant formula reduces, after polynomial expansion of the affine integrands, to an instance of a workhorse polynomial-degree-3 lemma whose closed form is $a_2/(\lambda t) + a_0$ (the odd-power moments m_1, m_3 vanish by parity, and $m_2 = 1/(\lambda t)$). The constant and m_2 -coefficient terms in κ_3 then cancel *algebraically* by `ring`. Single session, 323 lines in one new file (`Laplace/OneD/HarmonicAffineKappa3.lean`), zero `sorry`, zero `axiom`, zero `native_decide`.

The deliberation step converged on Candidate B (direct moment expansion, harmonic-specific) over Candidate A or C (both of which would have required factoring κ_3 multilinearity through a *Threepoint*-side perturbation-direction-independence lemma we don't yet have). GPT-5.5 Pro's flag of that API gap was the decisive input.

1 Setting

Tide 13 (cross-susceptibility derivative) and Tide 5 ($\kappa_3 = 0$ for the identity observable) closed a tight neighbourhood of the harmonic case in `Threepoint/Harmonic.lean`. Today's I2 tide (`tide/gibbscov-algebra`) added the generic algebra for `Laplace.gibbsExpectation` and `Laplace.gibbsCov` (scalar pulling, additivity, constant collapse), explicitly named “C1 — affine κ_3 for harmonic” as one of three strict-improvement follow-ups it unblocks. This tide closes that follow-up.

The seabed at the start of the tide was the I2 branch tip¹, with this Tide's worktree branched directly off that tip so the I2 algebra is available without an intermediate merge.

2 The thing formalised

The headline theorem is

`Laplace.OneD.kappa3_harmonic_affine_eq_zero :`

¹Branch `tide/gibbscov-algebra`, commit `b844d7a`; commit prefixes truncated to seven characters throughout this retrospective. The branch is itself off post-G4 `main` at `141997c`.

```

theorem kappa3_harmonic_affine_eq_zero
  {lam t : ℝ} (hlam : 0 < lam) (ht : 0 < t)
  (b1 c1 b2 c2 b3 c3 : ℝ) :
  Threepoint.kappa3 (volume : Measure ℝ)
    (harmonicPotential lam)
    (fun x : ℝ => b2 * x + c2)
  t
    (fun x : ℝ => b1 * x + c1)
    (fun x : ℝ => b3 * x + c3) = 0

```

`Threepoint.kappa3` is the third cumulant under a (L, A, t, h) -perturbed Gibbs measure, evaluated at $h = 0$ (`CrossSusceptibility.lean:49`); its argument order $(\mu, L, A, t, \varphi, B)$ is the convention from the threepoint paper, with A in the second observable position because A is also the *perturbation direction* of `gibbsExp`. At $h = 0$ that perturbation collapses, so mathematically κ_3 is fully symmetric in the three observable slots; in the Lean API only the swap $(\varphi \leftrightarrow B)$ is free (`kappa3_swap_φB`).

The *informal* statement is the multilinearity strict-improvement of Tide 5: κ_3 vanishes on any triple of affine observables, not just on the three identities. This is the cumulant-level expression of the fact that all Gaussians are determined by their first two moments (so cumulants of order three or higher vanish), and moreover that affine maps of a Gaussian are still Gaussian.

3 Proof strategy

The deliberation step considered three approaches:

Candidate A (multilinearity reduction). Introduce κ_3 -multilinearity lemmas in each slot, then specialise to $b_1 b_2 b_3 \cdot \kappa_3(\text{id}, \text{id}, \text{id}) = 0$ via Tide 5. Reusable infrastructure but ~ 150 – 250 lines.

Candidate B (direct moment expansion). Unfold κ_3 , bridge to `Laplace.gibbsExpectation` (taking advantage of I2’s algebra), expand each affine product as a degree-3 polynomial in x , substitute the harmonic moments, finish by `ring`.

Candidate C (private multilinearity). Same as A but the multilinearity helpers are kept private to the new file. Same line count as A.

GPT-5.5 Pro voted Candidate B and flagged a structural reason: in `Threepoint.kappa3`, slot A is also the perturbation direction of `gibbsExp`, so a true “ κ_3 is symmetric across all three slots” lemma in Lean would require an `h=0`-perturbation-independence step that doesn’t yet exist in the `Threepoint` repo. Candidates A/C *appear* cheaper than they are because that hidden API obligation was not folded into their estimates. Candidate B sidesteps the issue entirely: it works at the integrand level, where the permutation symmetry is mathematical (factoring through the `Threepoint`–`Laplace` bridge) rather than API-level.

The agreed proof is the moment-expansion route, with three layers of helpers in the same file:

Bridge. `threepoint_gibbsExp_volume_zero_eq` (private): for $\mu = \text{volume}$ on $\mathbb{R}$, `gibbsExp L A t 0 φ` reduces to `Laplace.gibbsExpectation L t φ`. Same private helper as in `AnharmonicKappa3.lean`; restated here to keep the file self-contained.

Moment closed forms. Three private wrapper lemmas computing $\langle x \rangle_t = 0$, $\langle x^2 \rangle_t = 1/(\lambda t)$, $\langle x^3 \rangle_t = 0$ from the existing `gibbsExpectation_harmonic_pow_*` lemmas in `OneD/Harmonic.lean`.

Polynomial-expectation workhorse. `gibbsExp_poly_le3_harmonic` (private): for any degree-3 polynomial coefficients (a_3, a_2, a_1, a_0) , the harmonic Gibbs expectation of $a_3x^3 + a_2x^2 + a_1x + a_0$ equals $a_2/(\lambda t) + a_0$. Path: `gibbsExpectation_add` three times (with type-ascribed integrability witnesses for each tail sum), `gibbsExpectation_smul` three times, `gibbsExpectation_const` once, then substitute the three moments.

The headline theorem is then a direct application of the workhorse to each affine product in the κ_3 expansion:

- For each pair-product $(b_i x + c_i)(b_j x + c_j)$, rewrite as $0 \cdot x^3 + b_i b_j \cdot x^2 + (b_i c_j + c_i b_j) \cdot x + c_i c_j$ via `funext`, then apply the workhorse to get $b_i b_j / (\lambda t) + c_i c_j$.
- For the triple-product $(b_1 x + c_1)(b_2 x + c_2)(b_3 x + c_3)$, rewrite as $b_1 b_2 b_3 \cdot x^3 + (\sum_{\text{sym}} b_i b_j c_k) \cdot x^2 + (\sum_{\text{sym}} b_i c_j c_k) \cdot x + c_1 c_2 c_3$ and apply the workhorse to get $(\sum_{\text{sym}} b_i b_j c_k) / (\lambda t) + c_1 c_2 c_3$.
- For the three single-affine expectations, apply `gibbsExp_affine_harmonic` to get the value c_i (since $\langle x \rangle = 0$).

The seven closed forms then plug into the cumulant formula, and `ring` closes the equality. The cancellation pattern is the one identified in the numerical sanity check: the $1/(\lambda t)$ -coefficient $(b_1 b_2 c_3 + b_1 c_2 b_3 + c_1 b_2 b_3) - b_1 b_2 c_3 - b_1 b_3 c_2 - b_2 b_3 c_1$ is identically zero by symmetry of multiplication, and the constant-coefficient $c_1 c_2 c_3 - 3c_1 c_2 c_3 + 2c_1 c_2 c_3$ also vanishes.

4 Roadblocks and resolutions

Two minor frictions, both anticipated by the laplace `CLAUDE.md`'s "Mathlib gotchas" section.

Pi.add vs single-lambda mismatch. `Integrable.add` returns its conclusion as $f + g$ in the `Pi.add` encoding, not as a single lambda $x \mapsto f(x) + g(x)$. Subsequent `Integrable.congr` matches fail because the goal sees `Pi.add` not the lambda. The fix is the standard idiom from `CLAUDE.md`:

```
filter_upwards with x
simp only [Pi.add_apply]; ring
```

i.e., unfold the `Pi.add` explicitly via `simp only [Pi.add_apply]` before `ring`. The first attempt (without the `simp`) hit *'typeclass instance problem is stuck* and *'show' tactic failed to match* — typical signatures of this issue.

`convert ... using 1; ring on a closed goal.` A 'convert' that lands a perfectly-aligned goal (because `simp` had already done the alignment) leaves the `ring` call with no goals. Replaced with `exact h`. Tiny but worth recording: when the `simp` normal form already matches, `convert` is wrong; `exact` is right.

No GPT consult was needed during Step 3: the deliberation in Step 2 was load-bearing enough that the proof skeleton was clear from the start. The implementation rolled through in a single pass once the integrability discipline (`Pi.add_apply` unfolding) was in place.

5 What was Mathlib, what was new

Mathlib lookups (load-bearing).

- `Integrable.const_mul`: scaling preserves integrability.
- `Integrable.add`: sum of integrables (used through `Pi.add_apply`-folded integrability witnesses).
- `Pi.add_apply`: the simp lemma that unfolds `Pi.add` to pointwise sum.
- `filter_upwards` for almost-everywhere equality witnesses passed to `Integrable.congr`.

laplace-side existing material reused.

- I2’s algebra: `Laplace.gibbsExpectation_smul`, `_add`, `_const` (`Laplace/Gibbs.lean`, post-41a7b41). Without I2, this proof would have had to unfold `gibbsExpectation` manually to ratios of integrals and re-do the linearity by hand.
- Partition-function positivity for the harmonic case², used to discharge the $Z \neq 0$ hypothesis on `gibbsExpectation_const`.
- Harmonic moment closed forms³: direct $m_1 = m_3 = 0$ and $m_2 = 1/(\lambda t)$.
- Polynomial-times-Gaussian integrability⁴, giving integrability of $x^n \cdot \exp(-(t \cdot L(x)))$ for any $n \in \mathbb{N}$.

New material in this tide. Three single-purpose moment lemmas, two integrability convenience wrappers, the `Threepoint-to-Laplace` bridge restated from `AnharmonicKappa3.lean`, the polynomial-expectation workhorse (`gibbsExp_poly_le3_harmonic`), and the headline.⁵ Total: 323 lines, of which ~ 110 are docstrings and ~ 210 are proof body. Nine named lemmas, one of them public (the headline).

Threepoint API deltas. None. The proof works inside the `Laplace` namespace using the bridge lemma; `Threepoint.kappa3` and `Threepoint.gibbsExp` appear only in the headline statement (by construction) and in a single `simp only` call at the top of the proof.

6 Lessons

The “API symmetry vs. mathematical symmetry” distinction. GPT-5.5 Pro’s flag on Candidates A/C was *the* decisive deliberation input. Both candidates’ line-count estimates (~ 150 – 250) were predicated on κ_3 being fully symmetric in (φ, A, B) at the API level — which it is mathematically, but *not* at the Lean signature level, because slot A is load-bearing for the `gibbsExp`

²`Laplace.OneD.harmonic_partition_pos` in `Laplace/OneD/HarmonicGibbsRegularity.lean`.

³`Laplace.OneD.gibbsExpectation_harmonic_pow_even` and `_pow_odd` in `Laplace/OneD/Harmonic.lean`.

⁴`Laplace.OneD.integrable_pow_mul_exp_neg_mul_sq` in `Laplace/OneD/IntegralRemainder.lean`, combined with the `harmonic_integrand_eq` alignment.

⁵Full list: `gibbsExp_x_harmonic`, `gibbsExp_x_sq_harmonic`, `gibbsExp_x_cube_harmonic`, `integrable_smul_x`, `integrable_const_harmonic`, `threepoint_gibbsExp_volume_zero_eq`, `gibbsExp_affine_harmonic`, `gibbsExp_poly_le3_harmonic`, and the public theorem `kappa3_harmonic_affine_eq_zero`.

perturbation direction. Adding the missing `h=0`-perturbation-independence lemma is genuinely useful future infrastructure, but it would have been an unbudgeted sub-tide. Worth carrying forward into other deliberation checklists: *when a candidate's proof sketch quietly invokes a mathematical symmetry, check whether the corresponding API symmetry is already formalised.*

Moment expansion plus I2 algebra is a clean pattern. The seven Gibbs expectations in the κ_3 expansion all reduce to instances of a single polynomial-in- x workhorse lemma. The path `gibbsExpectation.add` ($\times 3$) + `gibbsExpectation.smul` ($\times 3$) + `gibbsExpectation.const` ($\times 1$) + `ring` is mechanical once the integrability bookkeeping is discharged. Subsequent strict-improvements of harmonic-Gibbs identities (e.g. G2 anharmonic affine κ_3 , future affine κ_4) can lift the workhorse pattern wholesale, replacing only the per-potential moment lemmas. Worth promoting the polynomial-degree- n workhorse to a `lean-common` (or `laplace-side`) helper once the *second* consumer materialises (G2 is the natural candidate).

Build-cache sharing across worktrees. A practical infrastructure note: the new worktree's `.lake/packages` pointed initially at the canonical `lean/laplace/` clone, which had unbuilt Mathlib / Threepoint dependencies (because `lake exe cache get` had failed offline at `tide-worktree create` time). Redirecting the symlink to a sibling worktree's `.lake/packages` (in this case the `tide/gibbscov-algebra` worktree, which had a clean build) gave instant green builds. This is safe so long as the two worktrees agree on Mathlib / Threepoint pin (`lakefile.toml` and `lake-manifest.json`); a sibling worktree branched off the same `main` should always agree. Worth adding to the `tide-worktree create` script as a fallback when `cache get` fails.

7 Follow-ups

- *G2 — anharmonic affine κ_3 .* The mirror tide for the anharmonic potential $L(x) = (\lambda/2)x^2 + (\alpha/6)x^3$. Same proof skeleton with the harmonic moment lemmas swapped for their anharmonic counterparts; the workhorse polynomial-expectation lemma generalises cleanly. Estimated ~ 80 lines after the workhorse promotion.
- *Promote the polynomial-degree- n workhorse to `lean-common`.* Once G2 lands and demonstrates the second consumer, lift `gibbsExp_poly_le3_harmonic` to a generic `gibbsExp_poly_le3` (or `_le_n`) parameterised by the moment closed forms. Reusable for any 1D Gibbs measure with explicit moments. ~ 80 lines.
- *The factored-identity formulation `kappa3.affine_eq_factored`.* GPT noted in the deliberation that the factored statement $\kappa_3(b_i x + c_i)_{i=1,2,3} = b_1 b_2 b_3 \cdot \kappa_3(x, x, x)$ is the natural *cumulant content* (constants drop, scalings factor) and is valid for any 1D Gibbs measure with appropriate integrability — not just the harmonic case. Promoting it to a standalone theorem in `Laplace.Gibbs` (or in a new `Threepoint-side` helper file) would be the natural setup for G2 and any future affine- κ_3 tides.
- *Affine κ_4 for the harmonic Gibbs.* Once a `kappa4` definition lands in `Threepoint`, the mirror affine-vanishing theorem follows the same workhorse pattern with the polynomial-expectation lemma extended to degree 4 (closed-form moments $m_4 = 3/(\lambda t)^2$ from the existing even-power harmonic lemma).
- *$h = 0$ -perturbation-independence for `Threepoint.gibbsExp`.* The lemma flagged by GPT as missing during deliberation. Statement: `gibbsExp  $\mu$  L  $A_1$  t 0  $\varphi$  = gibbsExp  $\mu$  L  $A_2$  t 0`

φ for any two perturbation directions A_1, A_2 . Mathematically a one-liner (the perturbation factor collapses at $h = 0$), but in Lean would require an `integrand_eq`-style argument since the integrand literally contains A . Defer until a downstream tide actually needs the symmetry that the existing `kappa3_swap_φB` doesn't cover.

Acknowledgements

GPT-5.5 Pro contributed the deliberation-stage advisory at Step 2, including the API-incompleteness flag for Candidates A/C (the κ_3 -symmetry vs. `Threepoint.gibbsExp`-perturbation-direction issue) and the recommended moment-expansion implementation style. The full consult is preserved at `projects/primer/tide-log/gpt55_tide_kappa3_affine_harmonic_v1.md`.

This Tide branched off the `tide/gibbscov-algebra` tip (b844d7a), so it consumes I2's algebra directly without an intermediate merge. When both branches eventually land in `main`, the linear chain `main` $\rightarrow$ `tide/gibbscov-algebra` $\rightarrow$ `tide/kappa3-affine-harmonic` merges cleanly in that order.